

AdamOS: Design and Implementation of a Unix-Like 32-bit x86 Operating System from Scratch

Arterio Rodrigues

Advanced Operating Systems — Spring 2026

Abstract

AdamOS is a 32-bit x86 operating system built from scratch in C and NASM assembly, running under QEMU emulation with no reliance on any existing kernel or standard library. The system progresses from a 512-byte BIOS boot sector through protected-mode initialization, a bitmap physical frame allocator, per-process paging, a preemptive round-robin scheduler, a FAT16 filesystem over ATA PIO, a 24-syscall interface via int 0x80, PS/2 keyboard and mouse drivers, and a kernel-resident VBE windowing system with double-buffered compositing. This report describes the design and implementation of each subsystem in turn, documents how the project satisfies the course's OS-level scope and implementation-depth requirements, presents a code quality audit identifying ten correctness bugs including a critical scheduler quantum defect, and outlines a Phase 2 roadmap toward full Unix semantics.

Index Terms—operating systems, x86 protected mode, memory management, process scheduling, system calls, FAT16, VBE graphics, windowing system.

I. INTRODUCTION

Building an operating system from scratch is one of the few programming projects that forces a developer to confront every layer of the stack simultaneously: the CPU privilege model, hardware I/O, memory management, process isolation, and user-facing abstractions must all be designed and made to work together without the safety net of an existing runtime. AdamOS is a 32-bit x86 operating system written by a single developer with exactly that goal.

The project had three motivating objectives. The first was to build something genuinely usable — not a tutorial kernel that prints 'Hello, world' and stops, but a system with real process isolation, a persistent filesystem, a shell, and working user programs. The second was to gain a ground-up understanding of the abstractions that every programmer depends on: what actually happens when a program calls `read()`, how the CPU enforces the boundary between user and kernel space, and what a context switch looks like at the register level. The third was creative: joining the tradition of solo systems builders such as Terry Davis (TempleOS) who demonstrated that a single engineer can construct a coherent operating environment from bare metal [1].

The primary reference throughout development was the OSDev community wiki [2], which documents x86 bootstrapping, descriptor tables, ATA drivers, and paging. Assembly-level routines were developed with AI assistance (Claude Sonnet 4.6 [3]) to fill gaps where knowledge

was thin; all C code and high-level design decisions were the developer's own. The system is implemented in C with NASM assembly and builds with the i686-elf cross-compiler toolchain.

This report is organized as a guided tour of AdamOS. Section II maps the project against the course requirements. Section III introduces the hardware platform and toolchain. Section IV provides a system-wide overview. Sections V through X trace the path from power-on through boot, memory, processes, filesystem, syscalls, and the GUI. Section XI presents the code quality audit. Section XII describes the Phase 2 roadmap, and Section XIII concludes.

II. PROJECT REQUIREMENTS AND ALIGNMENT

The course project guidelines require an OS-level focus on core kernel mechanisms such as scheduling, memory management, synchronization, IPC, or filesystems; implementation depth (the mechanism must be built, not wrapped); correctness analysis with design trade-off discussion; and implementation in C or C++. Table I maps each requirement to specific AdamOS subsystems.

Requirement	AdamOS Implementation	Evidence
OS-level focus: scheduling	Preemptive round-robin scheduler driven by the 100 Hz PIT. Quantum management, PCB context switching, and process state transitions (READY/RUNNING/WAITING/ZOMBIE) are implemented from scratch in scheduler.c and process-control-block.c.	§VI, Fig. 3
OS-level focus: memory management	Bitmap physical frame allocator (frame.c, 128 MB / 32,768 frames); per-process two-level page directories with map_page(); first-fit kernel heap allocator (kmallocc.c, 3 MB). No external library used.	§V, Fig. 2
OS-level focus: IPC / synchronization	stdin ring buffer with a wait queue for blocking reads; waitpid() synchronization between parent and child; file descriptors as a shared resource managed across fork/exec.	§VI.D, §VII.C
OS-level focus: filesystem	FAT16 filesystem implemented in fat16.c: directory traversal, FAT chain following, file creation, and sector-level write. Accessed via ATA PIO driver (ata-disk.c) also written from scratch.	§VII
OS-level focus: device management	Drivers written from scratch for: PS/2 keyboard (IRQ1, scancode→ASCII), PS/2 mouse (IRQ12, 3-byte packets), PIT timer (100 Hz), ATA disk (LBA28 PIO), VGA text mode (ANSI parser), VBE pixel framebuffer (double-buffered).	§III, §IX
Implementation depth	Zero standard-library dependencies. The custom implementations include string/mem/math utilities, a bitmap allocator, a first-fit heap, a round-robin scheduler, a FAT16 driver, an ANSI terminal, and a VBE compositor — all written by hand.	Entire codebase
Language	Primary implementation language is C (i686-elf-gcc, -ffreestanding). Interrupt stubs, context switch, and	build-kernel.sh

Requirement	AdamOS Implementation	Evidence
	bootloader are in NASM x86 assembly. No C++ or Python.	
Correctness & analysis	Code review identified 10 correctness bugs (Table III). Design trade-offs are analyzed throughout (e.g., polling vs. DMA, monolithic vs. microkernel, eager copy vs. COW fork, first-fit heap). Limitations and performance characteristics are explicitly discussed.	§XI, §V–§X

Table I. Mapping of course project requirements to AdamOS subsystems.

AdamOS exceeds the minimum scope by implementing five distinct OS-level mechanisms simultaneously — scheduling, memory management, IPC, filesystem, and device drivers — all integrated into a system that boots, runs isolated user processes, and persists data across sessions. The windowing system and compositor, while beyond core OS scope, demonstrate additional depth in kernel-level resource management and interrupt-driven I/O.

III. PLATFORM AND TOOLCHAIN

AdamOS targets the Intel x86 32-bit (IA-32) instruction set. Protected mode, introduced with the 80386, provides the two features the kernel depends on most: hardware-enforced privilege rings (ring 0 for the kernel, ring 3 for user programs) and page-based virtual memory. Without these, process isolation and memory protection would require software workarounds that would fundamentally change the design.

The toolchain is a cross-compilation environment that produces freestanding IA-32 binaries. The C compiler is i686-elf-gcc with flags `-ffreestanding -fno-pic -include pch.h`; the linker is i686-elf-ld; the assembler is NASM (ELF32 for kernel objects, flat binary for the boot sector). A precompiled header (pch.h) aggregates all kernel includes so that every .c file needs only one include directive. No standard library is linked; string, memory, and math utilities are provided by custom implementations in lib/.

The build entry point is run.sh, which chains user-space/build-user.sh (compiles user programs into flat binaries) and kernel-space/build-kernel.sh (assembles the kernel, creates a FAT16 disk image with mkfs.fat and mcopy, and concatenates everything into a 10 MB os-image.bin). The FAT16 partition begins at LBA 2048; the kernel occupies the first ~50 sectors of the raw image.

IV. SYSTEM ARCHITECTURE

AdamOS is a monolithic kernel: all subsystems share a single ring-0 address space. There is no microkernel message passing, no loadable module infrastructure, and no user-space system server. Every service — scheduling, filesystems, drivers, the window manager — executes in ring 0. This design maximises simplicity and performance at the cost of fault isolation; a bug in any subsystem can corrupt the entire kernel. Eight subsystems compose the system, summarized in Table II.

Subsystem	Role
Bootloader	16-bit BIOS boot → disk load → GDT → protected mode transition
Memory Manager	Bitmap frame allocator (128 MB / 32,768 frames); per-process page directories; first-fit kmalloc heap (3 MB at 0x100000)
Scheduler	Round-robin; 100 Hz PIT; 20-tick quantum; PCB linked list; iret-based context switch
Syscall Interface	24 syscalls via int 0x80; EBX/ECX/EDX for args; EAX for return value
Filesystem	FAT16 on ATA PIO LBA28 disk; read, write, create, directory traversal
Drivers	VGA text (80×25, ANSI); VBE pixel framebuffer; PS/2 keyboard & mouse; PIT timer
Window Manager	Kernel-resident; linked-list of windows; z-ordering; double-buffered compositor
User Space	Ring-3 shell + programs (vim, brainfuck interpreter, idle); isolated virtual address spaces

Table II. AdamOS subsystem summary.

V. BOOT SEQUENCE

The boot sequence is the most mechanically constrained part of the system because the CPU starts in 16-bit real mode with almost no services available. Every step must be carefully ordered: switching to protected mode before loading the GDT, for example, results in an immediate triple fault. Fig. 1 shows the seven-stage pipeline from BIOS entry to the first user processes.

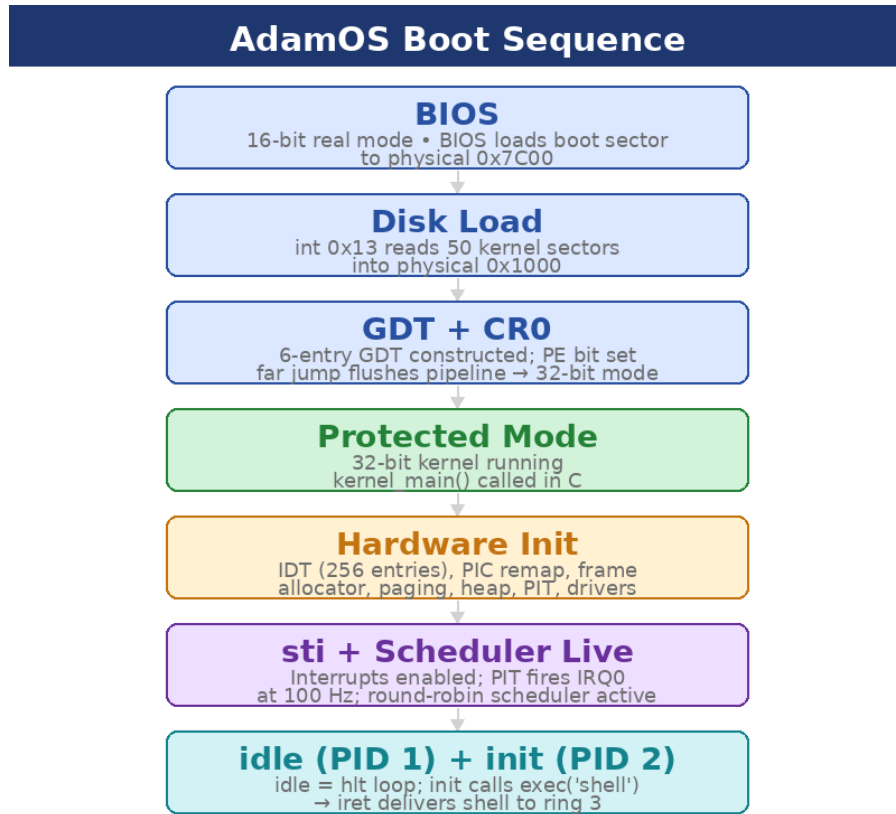


Fig. 1. AdamOS boot sequence from BIOS to first user processes.

A. Real Mode: Disk Load

The BIOS loads the first 512-byte sector of the disk image to physical address 0x7C00 and transfers control there in 16-bit real mode. The boot sector uses BIOS interrupt 0x13 (extended disk read) to load 50 sectors of kernel code to 0x1000. Fifty sectors was chosen empirically to cover the full kernel binary with comfortable headroom. The boot sector ends with the mandatory 0x55AA signature at byte offset 510.

B. Protected Mode Transition

Before switching modes, the bootloader constructs a six-entry Global Descriptor Table (GDT) in memory and loads it with the lgdt instruction. The six entries are: null, kernel code (ring 0), kernel data (ring 0), user code (ring 3), user data (ring 3), and a Task State Segment (TSS). The TSS entry is essential: when a ring-3 process issues int 0x80, the CPU reads TSS.ESP0 to find the kernel stack for that process. Setting the PE bit in CR0 and immediately executing a far jump flushes the pipeline and completes the transition to 32-bit protected mode.

C. Hardware Initialization

Control passes to kernel-entry.asm at 0x1000, which calls kernel_main() in C. kernel_main() initializes subsystems in strict dependency order. The Interrupt Descriptor Table (IDT) is

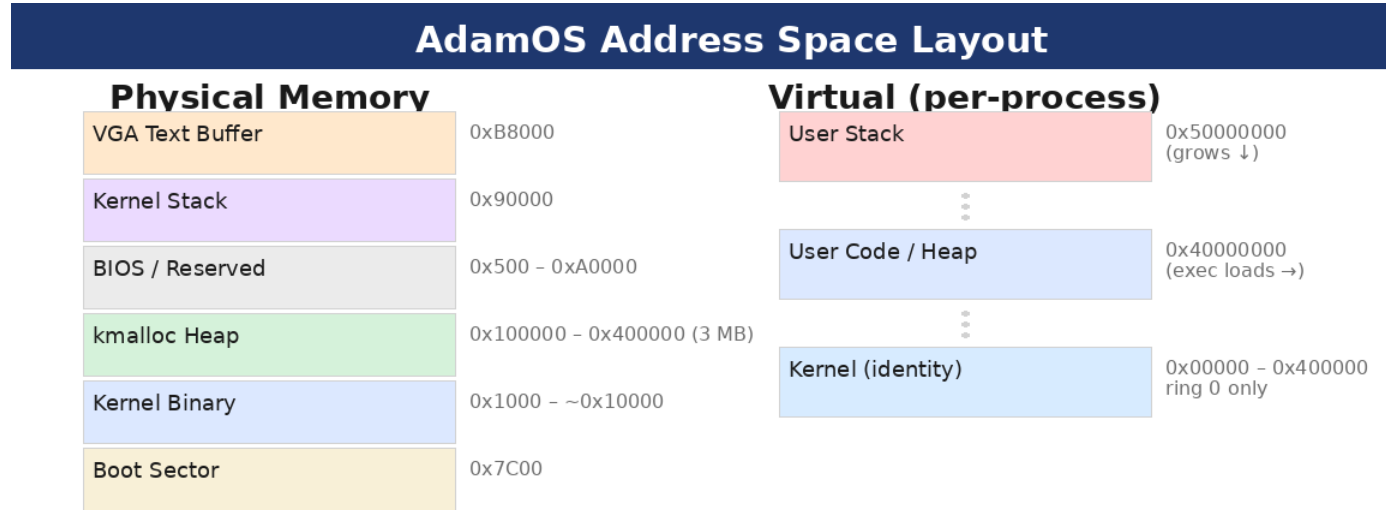
populated with 256 entries: CPU exceptions 0–19 are wired to individual handlers, and the eight PIC IRQs are remapped from their BIOS default range (0x08–0x0F) to 0x20–0x2F to avoid collisions with CPU exception vectors. The syscall gate at IDT entry 0x80 is given DPL=3, making it accessible from ring 3. The frame allocator and paging are initialized next, followed by the kmalloc heap, PIT at 100 Hz, and the VBE, PS/2 keyboard, and PS/2 mouse drivers. Interrupts are enabled last (sti) to guarantee all handlers are registered before the first IRQ fires.

D. First Processes

With interrupts live, `kernel_main()` creates two processes. PID 1 (idle) is a bare hlt loop that runs whenever no other process is READY, preventing the CPU from spinning on an empty scheduler. PID 2 (init) immediately calls `sys_exec('shell')`, which loads the shell binary from FAT16 into user-space virtual memory and performs an `iret` to ring 3. From this point, the system is fully operational: the PIT fires IRQ0 every 10 ms, the scheduler runs, and the user faces a shell prompt.

VI. MEMORY MANAGEMENT

Memory management in AdamOS operates at three levels: physical frame tracking, virtual address mapping per process, and dynamic kernel heap allocation. Fig. 2 shows how the address space is partitioned between the kernel and user programs.



Kernel region is identity-mapped in every process page directory (virtual address = physical address).

Fig. 2. Physical memory layout (left) and per-process virtual address space (right).

A. Physical Frame Allocator

`frame.c` manages physical memory as a flat bitmap of 32,768 bits, one per 4 KB page frame, covering 128 MB of physical RAM. `allocate_frame()` performs a linear scan for the first clear bit, sets it, and returns the corresponding physical base address; `free_frame()` clears the bit. The $O(n)$

scan is acceptable at current scale; a free-list would improve allocation to $O(1)$ but is deferred to Phase 2. A known bug (§XI, Bug 1.9) causes the startup reservation loop to allocate random frames rather than directly marking the kernel's physical pages in the bitmap.

B. Virtual Memory and Paging

Each process has its own page directory. `map_page()` installs a 4 KB virtual-to-physical mapping given a virtual address, a physical frame address, and a flags byte. The kernel region (physical 0x000000 to 0x4000000) is identity-mapped in every process's page directory — virtual equals physical — so kernel code executes identically regardless of which CR3 is loaded. User-space code lands at virtual 0x40000000 and the user stack at 0x50000000.

`fork()` deep-copies the parent's entire page directory: for every present user-space page, a new physical frame is allocated and the contents are copied. This is correct but expensive. Copy-on-write — marking pages read-only on fork and copying only on page fault — is the principal Phase 2 stretch goal and would make `fork()` effectively $O(1)$ in the common case.

C. Kernel Heap

The kernel heap starts at physical 0x100000 and spans 3 MB. Each allocation is preceded by a header recording block size and a used/free flag. `kmalloc()` performs a first-fit linear scan; `kfree()` marks the block free and coalesces adjacent free blocks eagerly. Because the kernel is single-threaded between interrupt handlers, no locking is required.

VII. PROCESS MANAGEMENT

Processes are the central abstraction of AdamOS. Understanding how they are created, scheduled, and terminated requires following a chain that touches the PCB, the scheduler, virtual memory, the filesystem, and the syscall interface. Fig. 3 shows the four process states and the events that move a process between them.

Process Lifecycle State Machine

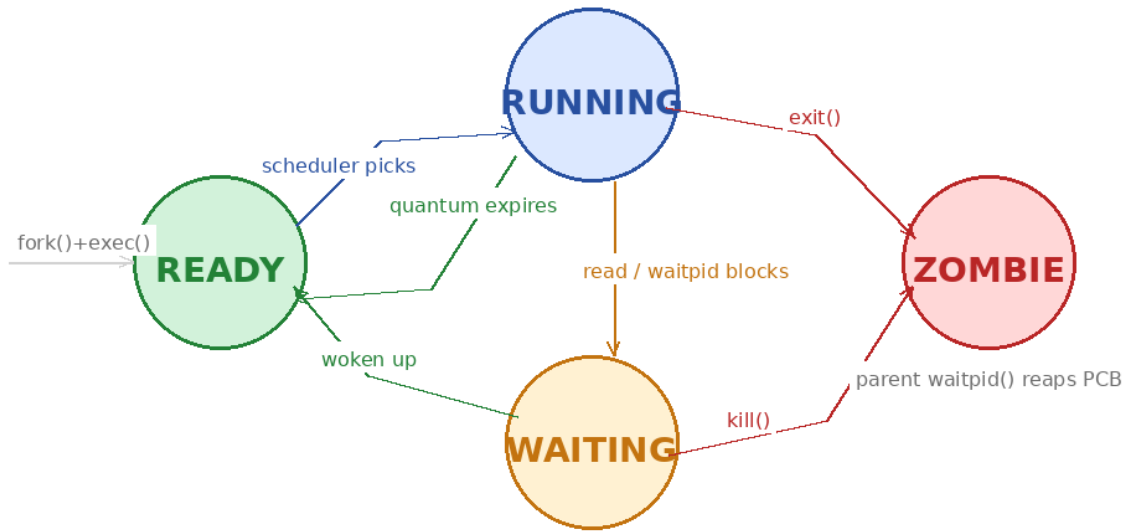


Fig. 3. AdamOS process lifecycle state machine.

A. The Process Control Block

Every process is represented by a `pcb_t` structure allocated from the kernel heap. It stores a saved register snapshot (`registers_t`, matching the pushad layout), the physical address of the process's page directory (loaded into CR3 when the process runs), a pointer to its kernel stack, PID and parent PID, a 16-slot file descriptor table, the current process state, and a `sleep_ticks` counter decremented by the timer ISR. The PCBs form a singly-linked list walked by the scheduler.

B. Round-Robin Scheduler

The PIT fires IRQ0 at 100 Hz. Each tick, `timer_handler_main()` decrements sleep counters (waking sleeping processes by transitioning them to READY) and increments the global quantum counter. When the counter reaches 20 — every 200 ms — `update_scheduler()` performs a context switch: it finds the next READY process, swaps CR3 to its page directory, updates TSS.ESP0 to its kernel stack top, copies the saved register snapshot, and executes `iret`, which atomically restores EIP, CS, EFLAGS, and ESP and returns to ring 3.

A critical bug (§XI, Bug 1.1) undermines this design: the scheduler guard compares the PCB pointer to the integer value of the RUNNING enum (which is 0), effectively testing `current_process == NULL`. The short-circuit intended to keep a running process on-CPU for its full quantum never fires, collapsing the effective quantum from 200 ms to 10 ms.

C. `fork()` and `exec()`

`fork()` duplicates the calling process. A new `pcb_t` is allocated, the parent's register snapshot is copied into it, the page directory is deep-copied, and a new kernel stack is allocated. The child's

saved `eax` is forced to zero; the parent's `eax` receives the child PID. Both PCBs enter the scheduler as `READY`.

`exec()` replaces the current process image. It locates the named binary on FAT16, frees all current user-space pages back to the frame bitmap, loads the flat binary at virtual `0x40000000` sector-by-sector via ATA PIO, resets `ESP` to `0x50000000`, and clears the register snapshot. An `iret` then delivers the process to ring 3 at offset 0. The fixed load address is the primary motivation for the ELF loader in Phase 2.

D. Blocking and `waitpid()`

When `sys_read()` is called on an empty `stdin` buffer, the kernel transitions the process to `WAITING` and enqueues it in a wait queue in `stdin.c`. The keyboard ISR wakes it (`READY`) when data arrives. `sys_waitpid()` similarly blocks the parent; `sys_exit()` in the child transitions it to `ZOMBIE` and wakes the parent. The current implementation does not encode exit status — a Phase 2 item.

VIII. FILESYSTEM AND I/O

A. ATA PIO Driver

Disk access uses ATA Programmed I/O through ports `0x1F0–0x1F7`. The driver writes the 28-bit LBA address across four ports, issues `READ SECTORS (0x20)`, polls the status register until `DRQ` is asserted, then transfers 512 bytes with `rep insw`. Writes append a cache-flush command (`0xE7`). Polling blocks the CPU for the full transfer duration — acceptable in QEMU but would be replaced with DMA interrupts in a production environment.

B. FAT16 Filesystem

The disk image is formatted as FAT16 (`mkfs.fat`) with user programs and data files copied in via `mcopy` before build. The FAT16 partition begins at LBA 2048. `fat16.c` implements: `fat16_find()` for directory traversal, `fat16_read()` for FAT chain following, `fat16_create()` for directory entry allocation, and `fat16_write()` for data and FAT update.

These functions are called directly from the syscall handlers — there is no indirection layer. Adding a second filesystem (or a device node like `/dev/null`) currently requires modifying syscall handlers directly. The VFS layer in Phase 2 decouples this coupling.

C. File Descriptors

Each process carries a 16-slot FD table in its PCB. FD 0 is `stdin` — a global ring buffer fed by the keyboard ISR, with a wait queue for blocking. FD 1 is `stdout`, wired to the VGA console. FDs 2–15 are allocated by `sys_open()` and reference FAT16 directory entries. `sys_poll()` provides non-blocking readiness checks.

IX. SYSCALL INTERFACE

The syscall interface is the sole bridge between ring-3 user programs and the ring-0 kernel. AdamOS provides 24 syscalls invoked by placing the number in EAX, arguments in EBX/ECX/EDX, and executing `int 0x80`. Fig. 4 traces the full path from the user instruction to the C handler and back.

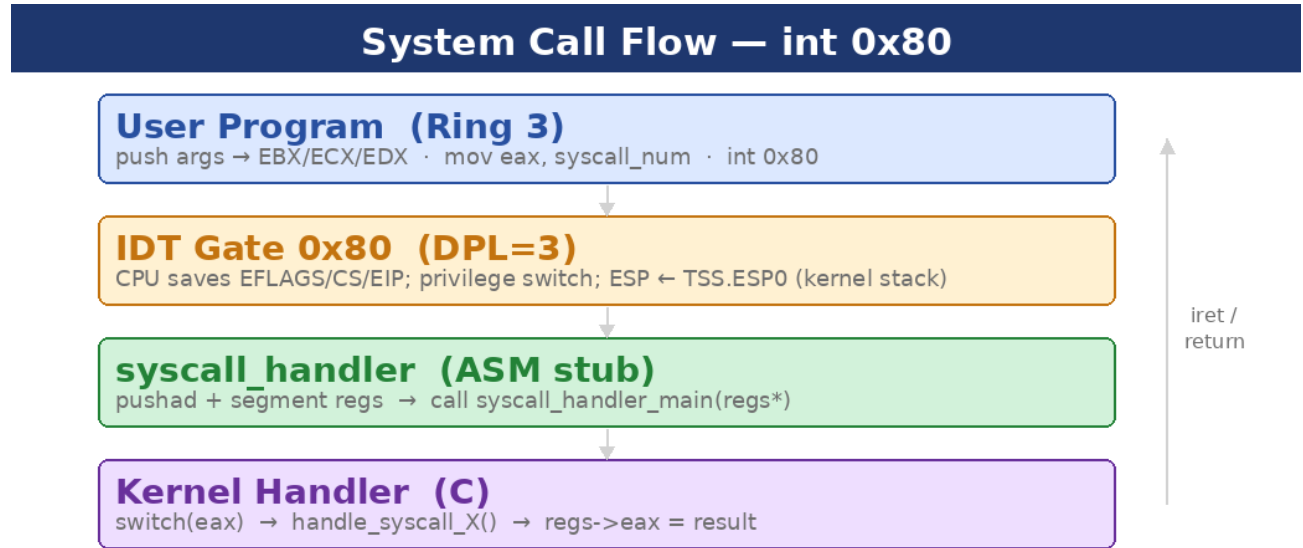


Fig. 4. System call execution path through IDT gate, ASM stub, and C dispatcher.

When `int 0x80` fires, the CPU reads TSS.ESP0 for the kernel stack, saves EFLAGS/CS/EIP, and jumps to the IDT gate handler. The ASM stub executes `pushad` and segment register saves, then calls `syscall_handler_main()` with a pointer to the register snapshot. The C dispatcher switches on EAX and calls the appropriate `handle_syscall_X()` function. Return values are written to registers `_t.eax`, and the complementary `popad/iret` restores user-space context. Table III groups the 24 syscalls by category.

#	Syscall	Description
1	exit	Terminate process; PCB transitions to ZOMBIE
2	fork	Deep-copy page directory; child receives <code>eax=0</code>
3	read	Read from fd; blocks on stdin when buffer empty
4	write	Write to fd; stdout triggers ANSI color parser
5	open	Locate file on FAT16; allocate fd slot
6	close	Mark fd slot free
7	ps	Copy process list (PID, state, PPID) to user buffer
11	exec	Load flat binary from FAT16 at 0x40000000; iret to ring 3
12	kill	Remove target PID from scheduler list
13	waitpid	Block caller until specified child exits
14	create	Allocate FAT16 directory entry and write initial data

#	Syscall	Description
15	poll	Non-blocking: return 1 if fd has data ready
16	sleep	Set sleep_ticks; scheduler skips process until zero
17	uptime	Return global tick counter
18–24	GUI calls	create_window, draw_rect, draw_text, get_event, destroy_window, flush, create_line

Table III. AdamOS syscall reference.

X. GUI AND WINDOW MANAGEMENT

Most hobby kernels stop at a text-mode console. AdamOS adds a pixel framebuffer, a kernel window manager, and a compositing pipeline — all in ring 0. The result is a system where user programs can create and interact with graphical windows through syscalls. Fig. 5 shows how input events and rendering commands flow through the system.

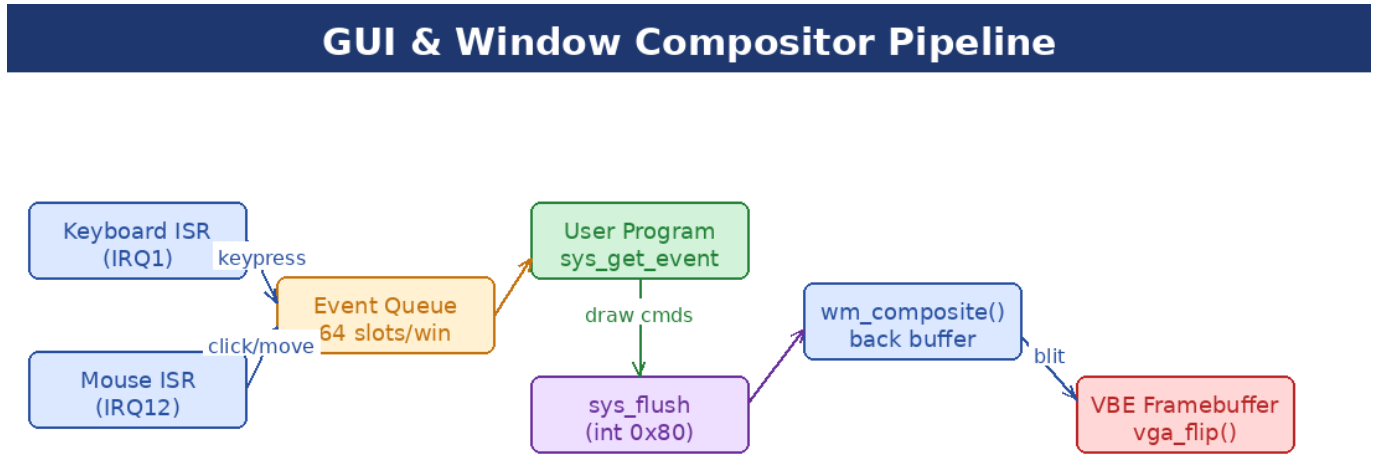


Fig. 5. GUI event and rendering pipeline from hardware ISRs to VBE framebuffer.

A. VBE Framebuffer

Before entering protected mode, the bootloader uses VESA BIOS Extensions to switch the display to a pixel-addressable mode and stores the mode-info block (framebuffer base address and resolution) at physical 0x500. vga-graphics.c reads this block during init. All drawing targets a back buffer (a kmalloc'd region); vga_flip() blits it to the VBE framebuffer atomically on sys_flush(), preventing the tearing that direct framebuffer writes would cause. A 5×7 bitmap font supports text in graphics mode.

B. Window Manager

window.c maintains a singly-linked list of window_t structures. Each window has its own pixel buffer, screen position, width, height, z-index, focus flag, drag state, and a ring buffer of 64 events. sys_create_window() allocates the pixel buffer from kmalloc and appends the window to the list.

wm_composite() renders windows back-to-front, painting the desktop background, each window's pixel buffer, title bar, and close button in z-order. The focused window is always drawn last.

C. Event System

Keyboard and mouse ISRs dispatch events into the focused window's ring buffer. event_queue_push() writes at the head; event_queue_pop() reads from the tail. User programs poll with sys_get_event(), which copies the oldest event into a caller-provided buffer. Mouse drag is detected in the mouse ISR by tracking button state across 3-byte PS/2 packets; when dragging, the focused window's position updates and wm_composite() redraws.

D. Design Trade-Off: Ring-0 Compositor

The window manager and compositor execute entirely in ring 0. This eliminates the complexity of a display-server IPC protocol and makes window syscalls fast (no context switch to a separate process). The trade-off is that a misbehaving draw call can corrupt kernel state, and compositing costs are paid in an interrupt-adjacent kernel context. A user-space compositor using shared-memory framebuffers is the architecturally correct long-term design but is deferred to future work.

XI. CODE QUALITY AUDIT

A systematic review of the AdamOS source identified seven correctness bugs, four naming inconsistencies, three duplicate-definition clusters, and seven functions that would benefit from decomposition. Table IV lists the correctness bugs by severity.

Sev.	Location	Description
MED	syscall.c – write	ANSI parser reads buf[i+1] without checking i+1 < len; reads one byte past the buffer on the final byte.
MED	syscall.c – draw	No NULL guard after get_window(); if the window was destroyed, the dereference causes a kernel page fault.
LOW	shell.c	After creating a new file, fd still holds -1 (open failed before create). sys_write(-1,...) writes to an invalid fd.
LOW	kernel.c	Frame allocation loop discards returned pointers; marks random frames used rather than the kernel's physical pages.
LOW	syscall.c – write	Writing to FD 0 (stdin) feeds the stdin ring buffer, corrupting pending reads for blocked processes.
INFO	syscall.c – close	is_open = false written twice (copy-paste). Harmless duplicate.
INFO	syscall.c – open	NULL filename check returns without setting regs->eax, leaving the return value undefined.

Table IV. Correctness bugs identified in code review, ordered by severity.

XII. FUTURE WORK: PHASE 2 UNIX ROADMAP

Phase 2 targets six subsystems that would give AdamOS the core behavioral contract of a Unix-like OS. They are ordered by dependency: signals and pipes depend only on the existing process model; VFS must precede devfs and procfs; ELF and TTY can proceed in parallel once VFS exists.

A. Unix Signals

The current kill syscall simply removes the target from the scheduler. A proper implementation needs a pending signal bitmask and signal mask per process, a handler table per signal (default/ignore/user function pointer), and delivery on return to user-space by redirecting EIP to a signal trampoline that calls sigreturn when done. Initial signals: SIGKILL (9), SIGTERM (15), SIGINT (2, from Ctrl+C), SIGSEGV (11, from page-fault handler), and SIGCHLD (17). The kill syscall gains a signal-number argument.

B. Pipes and I/O Redirection

A pipe syscall allocates a kernel ring buffer and returns two file descriptors. dup2 allows a child to redirect its stdin/stdout before exec. The shell gains pipe syntax (cmd1 | cmd2) by forking, connecting children with dup2, and exec-ing each command.

C. Virtual Filesystem Layer

A VFS interposes a function-pointer vtable between syscall handlers and filesystem drivers. A mount table maps path prefixes to vtable instances. devfs provides /dev/null, /dev/zero, /dev/console, and /dev/disk0. procfs exposes live process state at /proc/<pid>/status as read-only virtual files generated on demand.

D. ELF32 Loader

An ELF32 loader in exec() parses the ELF header, iterates PT_LOAD program headers, allocates pages at p_vaddr, loads segment data, zeroes BSS regions, and sets the entry point from e_entry. This eliminates the fixed 0x40000000 load address and makes standard i686-elf-gcc output directly executable.

E. TTY and Terminal Subsystem

A TTY layer provides line discipline: buffering, backspace, Ctrl+C → SIGINT, Ctrl+D → EOF, Ctrl+Z → SIGTSTP. The TTY is exposed through the VFS at /dev/tty0. A foreground process group field in TTY state ensures signals reach only the correct group.

F. Proper Exit Status and Process Tree

waitpid should return an encoded exit status (exit code in high byte, signal in low byte; WIFEXITED/WIFSIGNALED macros for user-space). Orphans should be reparented to PID 1. getpid, getppid, and sbrk syscalls round out the POSIX process model.

XIII. CONCLUSION

AdamOS demonstrates that a single developer can construct a functional operating system from a 512-byte boot sector to a graphical windowing system with isolated user processes in the span of one project cycle. The system implements five distinct core OS mechanisms — preemptive scheduling, virtual memory management, blocking I/O synchronization, a FAT16 filesystem, and device drivers for disk, keyboard, mouse, and display — all from scratch in C and NASM assembly with zero standard-library dependencies.

Phase 2 adds the six subsystems that elevate AdamOS from a functional hobby kernel to a Unix-like operating system: signals, pipes, a VFS layer, an ELF loader, a TTY subsystem, and proper exit-status encoding. With those in place, AdamOS will support the complete Unix programming model — fork, exec, pipe, signal, and wait — from an implementation whose every line can be traced back to first principles.

REFERENCES

- [1] T. Davis, "TempleOS," templeos.org, 2013. [Online]. Available: <https://templeos.org>
- [2] OSDev Wiki contributors, "OSDev Wiki," 2024. [Online]. Available: <https://wiki.osdev.org>
- [3] Anthropic, "Claude (Sonnet 4.6)," 2025. [Online]. Available: <https://www.anthropic.com>
- [4] L. Torvalds, "Linux kernel," GitHub, 2024. [Online]. Available: <https://github.com/torvalds/linux>
- [5] R. Cox, M. F. Kaashoek, and R. T. Morris, "xv6," MIT, 2022. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2022/xv6.html>
- [6] Intel Corporation, "Intel® 64 and IA-32 Architectures Software Developer’s Manual," vol. 3A, 2023.